

Mediapipe Gesture Tracking (PTZ)

1. Experiment Objective

The PTZ pan-tilt tracks the gesture position while recognizing the gesture class (finger-count code and gesture name) and displaying it in real time.

2. Experiment Principle

1. Key Terms

Term	Description
MediaPipe Hands	Google's open-source hand detection and landmark localization framework; detects 21 hand landmarks and supports gesture classification
Landmark 5 (index finger MCP)	The base metacarpophalangeal joint of the index finger; serves as the anchor for gesture tracking — the most stable position and suitable as the pan-tilt lock-on target
Gesture Recognition	Infers the gesture class (e.g., "Scissors", "OK") from the extension/bending state of the 5 fingers (<code>fingersup</code>)
Visual Tracking	Detects and continuously locks onto a target through a vision algorithm; only the pan-tilt follows, the car does not move
PTZ Pan-Tilt	A two-degree-of-freedom servo pan-tilt; s1 rotates horizontally and s2 tilts, controlled via the <code>/cmd_ptz_angle</code> topic
PD Control	Proportional-derivative control; computes the pan-tilt angle step from the target offset and the offset change rate
Deadband	When the target offset is within the deadband pixel range, the pan-tilt does not move, avoiding continuous jitter caused by tiny errors

2. Technical Architecture

ESP32 camera (firmware) → `/camera/image_raw` → `gesture_tracker_ptz_node` → `/cmd_ptz_angle` → firmware servo control

The state estimation layer (`ekf_filter_node` + `camera_node`) provides the image data stream

Node chain: state estimation (`ekf_filter_node` + `camera_node`) → `gesture_tracker_ptz_node` → `/cmd_ptz_angle` → firmware servo control

3. Core Topics Quick Reference

Firmware Uplink (data upstream):

Topic Name	Message Type	QoS	Description
/camera/image_raw	sensor_msgs/Image	BEST_EFFORT, KEEP_LAST depth=1, VOLATILE	Raw image stream from the ESP32 camera

Firmware Downlink (command downstream):

Topic Name	Message Type	QoS	Description
/cmd_ptz_angle	geometry_msgs/Vector3	RELIABLE, KEEP_LAST depth=10	Pan-tilt angle command (x=horizontal angle, y=tilt angle, z=reserved)

3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ version	☑
No-PTZ version	✗
Required peripherals	ESP32 camera (pan-tilt) + PTZ servo

2. Start Agent + Camera + Joint Model (Terminal 1)

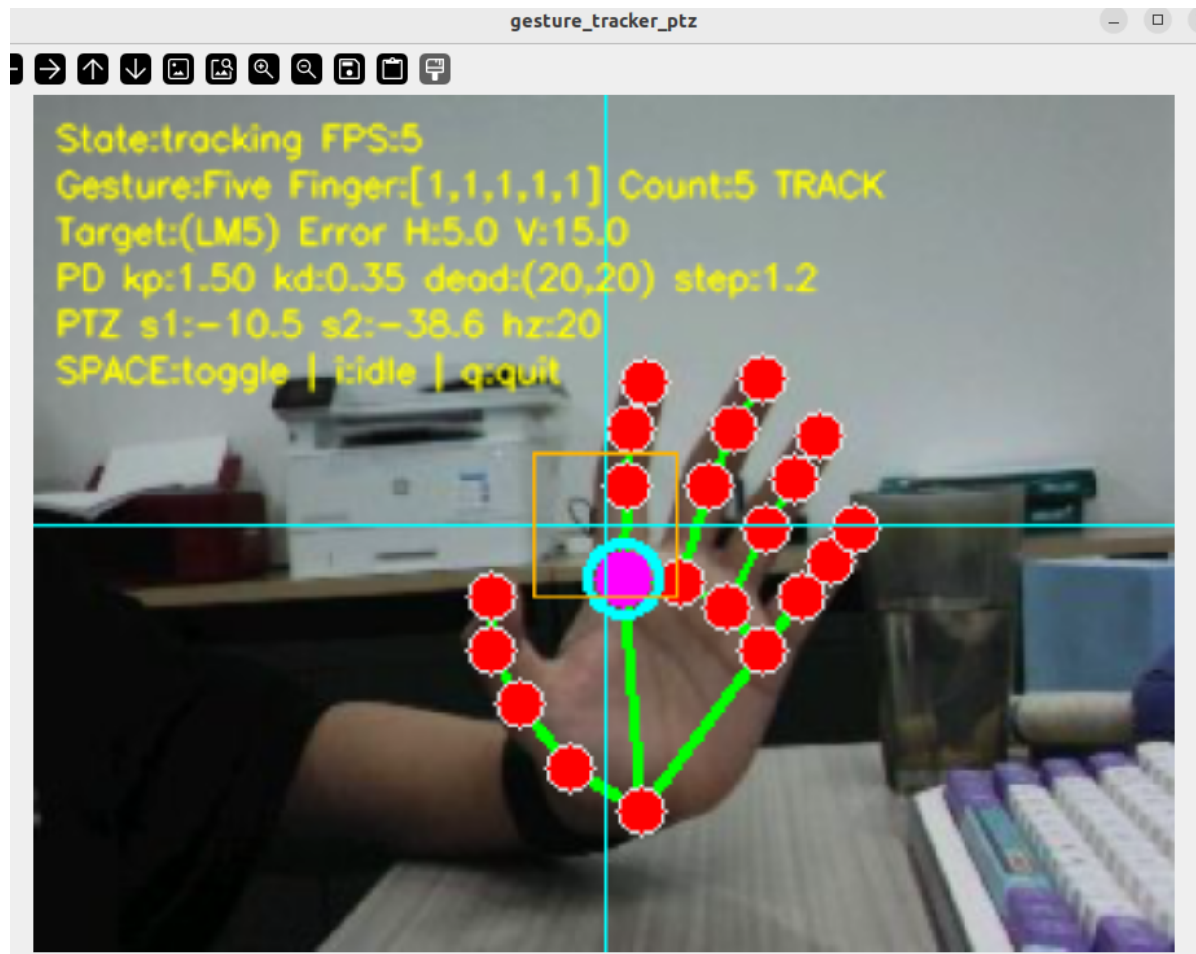
```
ros2 launch microros_v2_bringup car_base.launch.py
```

```
yahboom@yahboom:~$ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-28-15-04-46-269642-yahboom-579370
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [579411]
[INFO] [camera_driver-2]: process started with pid [579413]
[INFO] [robot_state_publisher-3]: process started with pid [579415]
[INFO] [joint_state_publisher-4]: process started with pid [579417]
[INFO] [ekf_node-5]: process started with pid [579419]
[micro_ros_agent-1] [1785222287.404236] info | UDPv4AgentLinux.cpp | init | running... | port: 8090
[robot_state_publisher-3] [INFO] [1785222288.847816851] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785222288.848013042] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785222288.848028942] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848037122] [robot_state_publisher]: got segment cam1_Link
[robot_state_publisher-3] [INFO] [1785222288.848043874] [robot_state_publisher]: got segment cam2_Link
[robot_state_publisher-3] [INFO] [1785222288.848050538] [robot_state_publisher]: got segment cam3_Link
[robot_state_publisher-3] [INFO] [1785222288.848057264] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785222288.848063747] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785222288.848070240] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848076888] [robot_state_publisher]: got segment rfwheel_Link
[joint_state_publisher-4] [INFO] [1785222289.372349515] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description
[camera_driver-2] [INFO] [1785222289.716161193] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[camera_driver-2] [WARN] [1785222292.705741414] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785222294.072701944] [esp32_ip_camera_publisher]: IP camera stream opened successfully
```

3. Start Gesture Tracking (Terminal 2)

```
ros2 launch microros_v2_ptz_visual_control_pkg gesture_tracker_ptz.launch.py
```

After startup, the frame shows the `idle` state; palm detection and gesture recognition run normally but no pan-tilt control is output.



4. Operation Instructions

1. Workflow

After startup, the node uses two independent timers: image processing (`image_processing_rate_hz` = 15 Hz) + pan-tilt control (`control_rate_hz` = 20 Hz). Minimal two-state machine: `idle` ↔ `tracking`.

(1) Image reception and preprocessing

Subscribes to `/camera/image_raw` (QoS: BEST_EFFORT, KEEP_LAST); the image callback converts and caches the raw Image. The image processing timer takes a copy of the latest frame, scales it to the processing resolution (`processing_width` × `processing_height` = 320×240), and flips/rotates as needed.

(2) MediaPipe palm detection + gesture recognition

Uses `HandDetector` (`detectionCon=hand_detection_confidence=0.8`) to detect the palm and extract:

- The 21 hand landmark coordinates (`lmList`)
- `fingersup()`: a 5-bit finger extension state list, 0=bent / 1=extended
- `finger_count`: the total number of extended fingers

- `get_gesture()` : the gesture name (e.g., "Scissors", "OK", "Thumbs Up", etc.)
- `finger_code` : the finger code string (e.g., "[0,1,1,0,0]")

(3) Tracking anchor extraction

Takes landmark 5 (index finger MCP joint) as the tracking target point. Draws the tracking point: a purple solid circle (radius 8 px) + a yellow hollow circle (radius 10 px) + the MediaPipe hand landmarks and connections.

(4) PD pan-tilt control (tracking + hand present + not a fist only)

Fixed-rate 20 Hz PD control: tracking point pixel coordinates → normalized error → PD step → limiting ($\pm \text{max_ptz_step} = 1.2^\circ$) → step smoothing ($\alpha = \text{ptz_step_smooth_alpha} = 0.35$) → direction reversal (horizontal not reversed, tilt reversed) → angle smoothing ($\alpha = \text{ptz_smooth_alpha} = 0.85$) → clamp the angle → publish.

When there is no hand or a fist (`fc==0`), no control is output, but the state is not changed and the PD history is not reset — tracking resumes seamlessly when the palm opens again.

(5) Gesture data display

The gesture name, finger code, and finger count are overlaid in real time at the top-left of the frame; "TRACK" is shown in the TRACKING state and "FIST" when a fist is detected.

2. Interaction

Key	Function	Description
Space	idle ↔ tracking	Toggles between standby and tracking
i / I	Back to idle	Stops tracking and pan-tilt control
q / Q / ESC	Exit	Force-exits the node process with <code>os._exit(0)</code>

On-screen overlay: The debug window shows the state (idle/tracking), frame rate, gesture name, finger code, finger count (with the FIST marker), tracking point error H/V, PD parameters (kp/kd/deadband/step), pan-tilt angle, and control rate. A crosshair is drawn at the frame center + an orange deadband rectangle.

Optional legend: With `overlay_show_legend=true`, the "SPACE:toggle | i:idle | q:quit" key hint is shown.

3. Safety and Edge Cases

- **No hand/fist, no control:** When there is no hand or a fist, the PD outputs nothing, but the state is not changed and the PD history is not reset, for a seamless handoff when tracking resumes
- **Gesture detection never changes the state:** Only the Space and i keys switch states; the detection result only affects whether control is output
- **Pan-tilt angle limiting:** Horizontal $[-90^\circ, 90^\circ]$, tilt $[-90^\circ, 20^\circ]$
- **Deadband protection:** The step is forced to zero when the tracking point offset is within the deadband (20 px)
- **Initial PTZ publish at startup:** With `publish_initial_ptz_command=true`, the initial angle ($0^\circ, -45^\circ$) is published 20 times at 0.2 s intervals
- **Exit strategy:** Press q/ESC to exit directly with `os._exit(0)`
- **Runtime parameter validation:** Range checks cover all PD/angle/image parameters; illegal values are rejected; `image_topic` / `ptz_command_topic` / `window_name` require a node restart

5. How to Stop

Press `Ctrl+C` in sequence to stop the processes in the three terminals: Terminal 2 (gesture tracking) → Terminal 1 (chassis, agent, camera, and model).

4. Experiment Observations

- **idle state:** The frame streams in real time; palm detection and gesture recognition run normally but no pan-tilt control is output, and the state is `idle`
- **tracking + hand present + fingers extended:** The pan-tilt follows the index finger MCP joint with PD control, the MediaPipe landmarks and connections are overlaid, and the gesture name and finger code update in real time
- **tracking + fist:** The frame shows "FIST", the pan-tilt stays still at its current position, and the PD history is kept; tracking resumes automatically when the palm opens again
- **tracking + palm moved out of frame:** The gesture display changes to "None", the pan-tilt stays at its last position, and tracking resumes when the palm reappears
- **Palm within the deadband:** The step is zeroed and the pan-tilt stays completely still without jitter
- **Gesture changes:** The gesture name updates in real time when different gestures are made, and tracking is unaffected (gesture detection and pan-tilt control are fully decoupled)
- **Press i to return to idle:** The pan-tilt stays at its current position; press Space to re-enter tracking

5. Program Analysis

Source code paths:

- Launch file:
`~/microros_ws/src/microros_v2_ptz_visual_control_pkg/launch/gesture_tracker_ptz.launch.py`
- Node file:
`~/microros_ws/src/microros_v2_ptz_visual_control_pkg/microros_v2_ptz_visual_control_pkg/gesture_tracker_ptz_node.py`
- Hand detection module:
`~/microros_ws/src/microros_v2_ptz_visual_control_pkg/microros_v2_ptz_visual_control_pkg/hand_detector.py`

1. Core Node Analysis

The node `GestureTrackerPtzNode` inherits from `rcipy.node.Node` and adds the full `get_gesture()` gesture classification logic on top of `HandDetector`, so it can recognize and display multiple gesture classes.

Constructor `__init__` core logic:

```
def __init__(self) -> None:
    super().__init__(DEFAULT_NODE_NAME)
    self._image_lock = threading.Lock()
    self._gesture_lock = threading.Lock()

    # State: IDLE / TRACKING only
    self._state = STATE_IDLE

    # Gesture shared data
```

```

self._has_hand = False
self._finger_count = 0
self._gesture_name = "None"
self._finger_code = "[0,0,0,0,0]"
self._target_x: Optional[float] = None
self._target_y: Optional[float] = None

# PTZ angles (initial 0°, -45°)
self._h_deg = 0.0
self._p_deg = 0.0

# PD history state
self._prev_err_x = 0.0
self._prev_err_y = 0.0
self._last_sx = 0.0
self._last_sy = 0.0

self._declare_params()
self._load_params()
self._h_deg = float(self._init_h_deg)
self._p_deg = float(self._init_p_deg)

# Initialize the MediaPipe gesture detector (with the confidence parameter)
self._hand_det = HandDetector(detectorCon=float(self._hand_conf))

# Image subscription (BEST_EFFORT, KEEP_LAST depth=1, VOLATILE)
self._img_sub = self.create_subscription(
    Image, self._img_topic, self._on_image,
    QoSProfile(history=QoSHistoryPolicy.KEEP_LAST, depth=1,
                reliability=QoSReliabilityPolicy.BEST_EFFORT,
                durability=QoSDurabilityPolicy.VOLATILE),
)

# Pan-tilt angle publishing
self._ptz_pub = self.create_publisher(Vector3, self._ptz_topic, 10)

# Image processing timer (15 Hz)
self._img_timer = self.create_timer(
    1.0 / max(float(self._img_rate), MINIMUM_TIMER_RATE_HZ),
    self._process_image,
)

# Pan-tilt control timer (20 Hz)
self._ctrl_timer = None
self._rebuild_ctrl_timer(False)

# Initial PTZ timed publishing (20 times, 0.2 s interval each)
if self._pub_init_ptz:
    self._start_init_ptz()

```

MediaPipe gesture detection `_detect`:

```

def _detect(self, frame: np.ndarray) -> None:
    self._hand_det.findHands(frame, draw=True)

    if len(self._hand_det.lmList) == 0:
        with self._gesture_lock:
            self._has_hand = False

```

```

        return

    # Extract landmark 5 (index finger MCP) as the tracking target
    mx, my = self._hand_det.findPoint(LANDMARK_INDEX_FINGER_MCP)

    # Finger extension state analysis
    fingers = self._hand_det.fingersUp()
    nf = [1 if int(v) != 0 else 0 for v in fingers]
    fc = int(sum(nf))
    fcode = "[" + ",".join(str(v) for v in nf) + "]"

    # Gesture class recognition
    gn = str(self._hand_det.get_gesture())

    # Draw the tracking point
    cv.circle(frame, (int(mx), int(my)), 8, (255, 0, 255), -1)
    cv.circle(frame, (int(mx), int(my)), 10, (255, 255, 0), 2)

    # Update the shared data
    with self._gesture_lock:
        self._has_hand = True
        self._target_x = float(mx)
        self._target_y = float(my)
        self._finger_count = fc
        self._finger_code = fcode
        self._gesture_name = gn

```

Gesture recognition and pan-tilt control decoupling design:

The key difference from the palm tracking node is that the gesture tracking node additionally displays the following data in the on-screen overlay:

```

State:tracking FPS:15
Gesture:Scissors Finger:[0,1,1,0,0] Count:2 TRACK
Target:(LM5) Error H:12.3 V:-5.6
PD kp:1.50 kd:0.35 dead:(20,20) step:1.2
PTZ s1:2.3 s2:-42.1 hz:20
SPACE:toggle | i:idle | q:quit

```

The `get_gesture()` method maps the 5-bit `fingersup()` code to a gesture name:

Finger Code	Gesture Name	Description
[0,0,0,0,0]	FIST	Fist (no pan-tilt control output)
[1,0,0,0,0]	Thumbs Up	Thumbs up
[0,1,0,0,0]	Point	Pointing with the index finger
[0,1,1,0,0]	Scissors	Scissors gesture
[1,1,1,1,1]	Open Palm	Open palm
[0,1,0,1,0]	Yo-Yo	Rock gesture
[0,1,1,1,0]	Three	Three fingers
[1,0,1,1,1]	OK	OK gesture

The PD control logic `_ctrl_tick` is identical to the palm tracking node, so it is not repeated here.

2. Key Parameters

Parameter definition file:

```
~/microros_ws/src/microros_v2_ptz_visual_control_pkg/launch/gesture_tracker_ptz.launch.py
```


Parameter	Type	Default Value	Description
<code>pd_kp</code> / <code>pd_kd</code>	float	1.5 / 0.35	PD proportional/derivative gains
<code>control_rate_hz</code>	float	20.0 Hz	Pan-tilt control rate
<code>image_processing_rate_hz</code>	float	15.0 Hz	Image processing rate
<code>deadband_x</code> / <code>deadband_y</code>	float	20.0 / 20.0 px	Horizontal/vertical deadband
<code>max_ptz_step</code>	float	1.2 °	Maximum single angle step
<code>ptz_step_smooth_alpha</code>	float	0.35	Step EMA smoothing coefficient
<code>ptz_smooth_alpha</code>	float	0.85	Pan-tilt angle EMA smoothing coefficient
<code>hand_detection_confidence</code>	float	0.8	Palm detection confidence threshold
<code>processing_width</code> / <code>processing_height</code>	int	320 / 240 px	Processing resolution
<code>display_width</code> / <code>display_height</code>	int	960 / 720 px	Display resolution
<code>initial_horizontal_deg</code> / <code>initial_pitch_deg</code>	float	0.0 ° / -45.0 °	Initial pan-tilt angles

6. Frequently Asked Questions

Problem	Cause	Solution
Gesture misjudgment makes the pan-tilt move erratically	Gesture recognition and pan-tilt control are decoupled by design — detection does not change the state, it only affects whether control is output	Confirm the operation logic: PD control is output only in tracking + hand present + not a fist
Palm lost when moving fast	Image processing at 15 Hz + PD response delay	Slow down the palm movement, or run <code>ros2 param set /gesture_follow_ptz_node pd_kp 0.9</code> to increase the proportional gain
Gesture classification uncertain	The finger angles are in an ambiguous region	Keep a standard gesture facing the camera and adjust the palm orientation so the fingers are clearly separated
No display after startup	The MediaPipe environment script was not sourced	First run <code>source /opt/ros/humble/setup.bash</code> <code>source ~/microros_ws/install/setup.bash</code>